

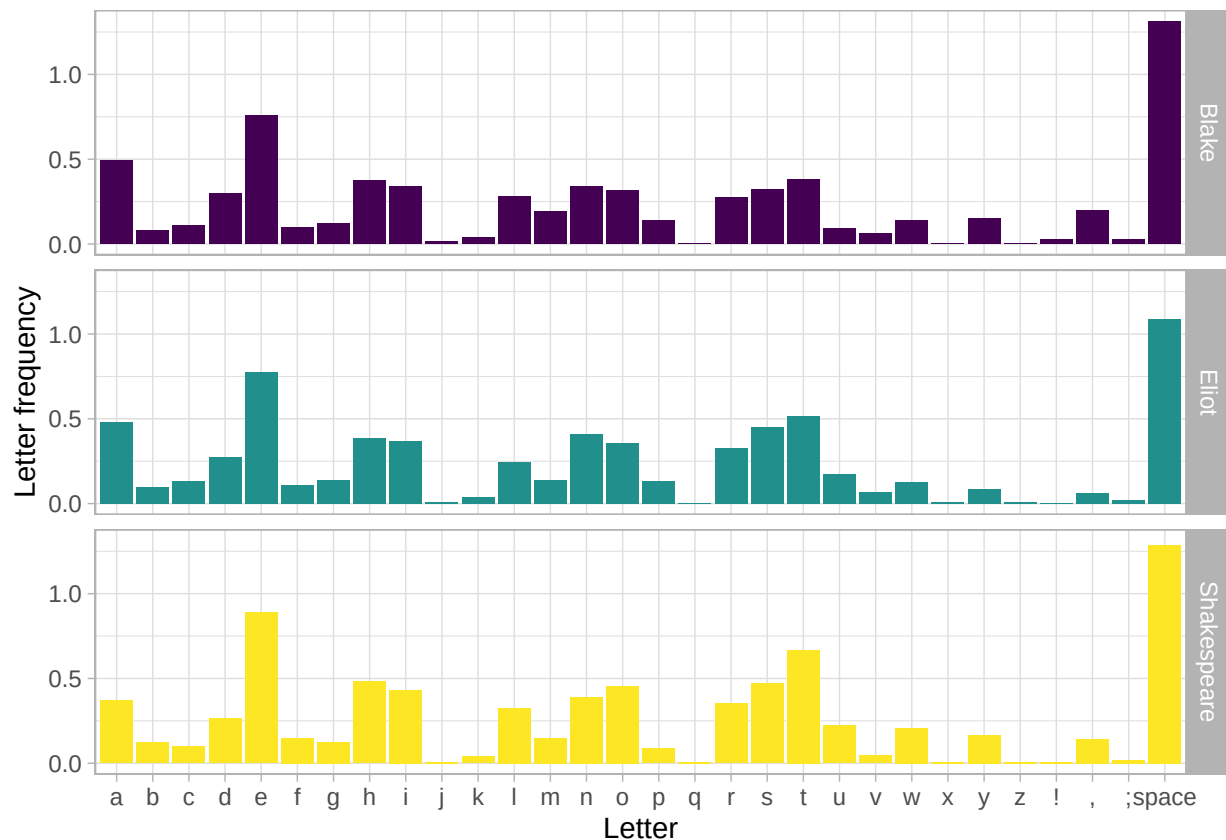
Proposed solution to STAT340 - Group Discussion 7

Classification

Ida Marie Munthe Sakseide

Exercise 1 - Analysis of ‘Deconstructed’ Poems by PCA

- a) Get acquainted with the data by visualizing letter frequencies in a bar plot. Compare the different authors.



- b) Conduct a principal component analysis using the `poem` data. What portion of the total variance in the data is captured jointly by the two first components? How many principal components are needed to explain 80 % of the variability?

```
pca <- prcomp(poems[, -1], scale=TRUE)
summary(pca)
```

```
## Importance of components:
```

```
##          PC1    PC2    PC3    PC4    PC5    PC6    PC7
## Standard deviation  2.4158 2.0752 1.8745 1.60611 1.52344 1.41857 1.27450
## Proportion of Variance 0.1945 0.1436 0.1171 0.08599 0.07736 0.06708 0.05415
## Cumulative Proportion 0.1945 0.3381 0.4552 0.54119 0.61856 0.68564 0.73978
##          PC8    PC9    PC10    PC11    PC12    PC13    PC14
## Standard deviation  1.1988 1.10033 1.06592 0.99768 0.95035 0.73060 0.60864
## Proportion of Variance 0.0479 0.04036 0.03787 0.03318 0.03011 0.01779 0.01235
## Cumulative Proportion 0.7877 0.82804 0.86591 0.89909 0.92920 0.94699 0.95934
##          PC15    PC16    PC17    PC18    PC19    PC20    PC21
## Standard deviation  0.58245 0.53702 0.47092 0.37338 0.3464 0.27495 0.18844
## Proportion of Variance 0.01131 0.00961 0.00739 0.00465 0.0040 0.00252 0.00118
## Cumulative Proportion 0.97064 0.98026 0.98765 0.99230 0.9963 0.99882 1.00000
##          PC22
## Standard deviation  6.298e-16
## Proportion of Variance 0.000e+00
## Cumulative Proportion 1.000e+00
```

PC1 and PC2 explain only about 34 % of the total variability in data, which is not much for making any decisive conclusions. We need the first 9 principal components in order to explain 80 % of the data variability.

- c) Which variable (letter/character) has the largest influence (loading) on PC1 (have a look at `pca$rotation`)? Which variable is most influential on PC2?

For simplicity, we will arrange the loading values for PC1 and PC2 in decreasing order by their absolute values.

```
df <- as.data.frame(pca$rotation[,1:2])
df$letter <- rownames(df)
rownames(df) <- 1:nrow(df)
df[,c(1,3)] %>% arrange(desc(abs(PC1)))
```

```
##          PC1 letter
## 1 -0.325537717  space
## 2  0.315930395    t
## 3  0.298844028    u
## 4 -0.254624407    !
## 5  0.254066101    s
## 6 -0.253268506    a
## 7 -0.250742672    m
## 8 -0.240556547    ,
## 9  0.230436756    f
## 10 0.188313509    x
## 11 -0.180659814   ;
## 12 0.178035516    b
## 13 0.170323532    z
## 14 0.165256732    r
## 15 -0.165241033   j
## 16 0.160193024    o
## 17 -0.146555104    p
## 18 -0.144285169    d
## 19 -0.141170810    y
## 20 0.139521413    e
## 21 0.125146778    h
```

```
## 22  0.082421362      q
## 23  0.080794790      w
## 24  0.075679781      i
## 25 -0.070146411      v
## 26  0.064806320      n
## 27 -0.046930053      g
## 28 -0.020835425      l
## 29 -0.015968848      c
## 30 -0.006309423      k
```

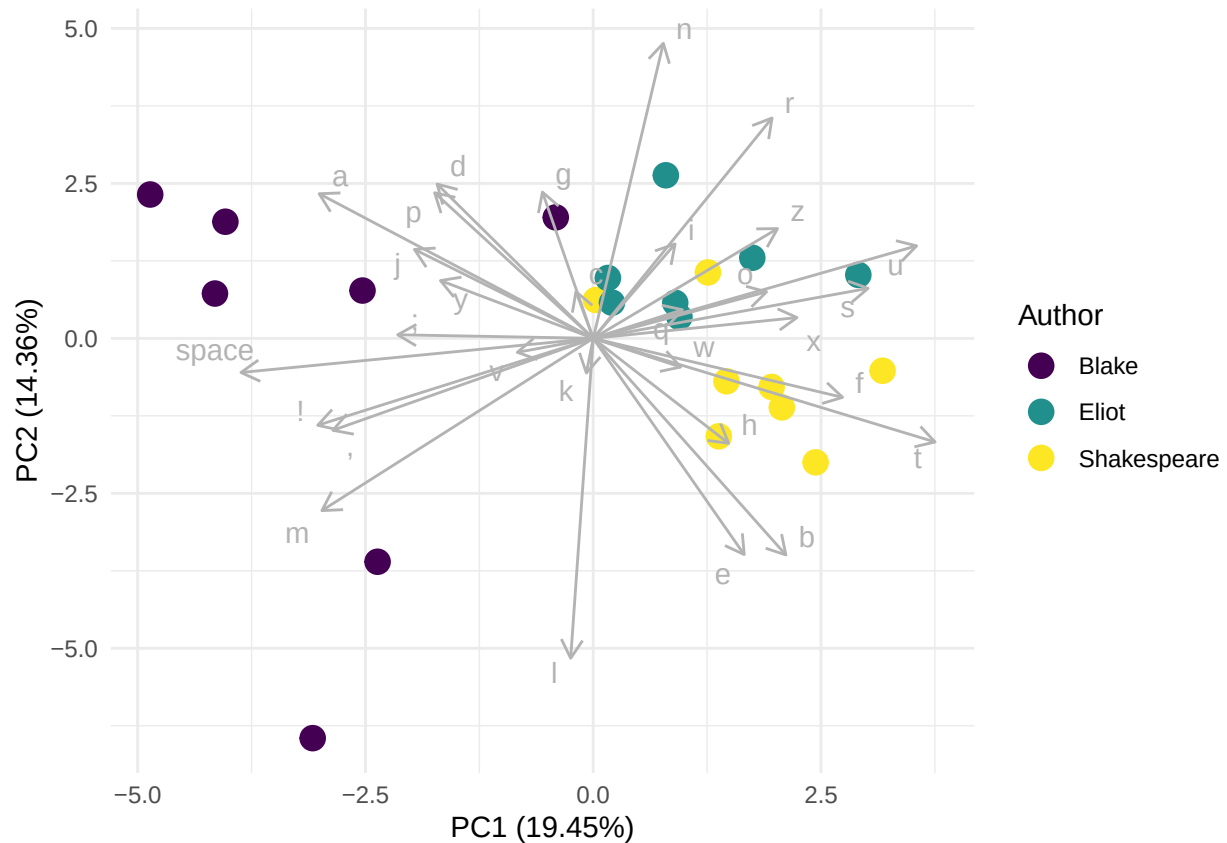
```
df[,c(2,3)] %>% arrange(desc(abs(PC2)))
```

```
##          PC2 letter
## 1 -0.434466407      l
## 2  0.400706345      n
## 3  0.299273777      r
## 4 -0.294185291      b
## 5 -0.293703198      e
## 6 -0.233939009      m
## 7  0.209539408      d
## 8  0.198653027      g
## 9  0.197756223      p
## 10 0.196412919      a
## 11 0.149073572      z
## 12 -0.142602016     h
## 13 -0.140804647     t
## 14 0.128183308      i
## 15 0.125626253      u
## 16 -0.124890499     ,
## 17 0.120811010      j
## 18 -0.118077385     !
## 19 -0.080147745     f
## 20 0.078457265      y
## 21 0.067931463      s
## 22 0.063385440      c
## 23 0.063209422      o
## 24 -0.047089499     k
## 25 -0.046404629    space
## 26 -0.038977466     w
## 27 0.038835500      q
## 28 0.028217576      x
## 29 -0.019017787     v
## 30 0.004866851      ;
```

For PC1, “space” has the largest influence (about -0.33). It’s loading is negative, meaning that authors that use a lot of space in their poems end up with a low PC1 score. Other characters have almost equally large influence in PC1, namely “t” and “u”. However, they have a positive loading, meaning that authors that use a lot of t’s and u’s end up with a large PC1 score.

For PC2, the letters “l” and “n” have the largest influence, and has noteworthy larger loading compared to the other characters.

- d) Create a biplot and comment (*NB: Discuss the output, not the code*). Do you think PC1 and PC2 are good variables for discriminating between the three authors? Justify your answer.



The biplot combines the information from a scoreplot and the loadingplot.

We see that poems written by Blake tend to be in the left half of the biplot. We infer from the biplot that poems of Blake contain more characters such as for instance space, exclamation mark (!), “a” and “m”. There is little overlap between poems of Blake and poems of the other authors.

Poems written by Eliot tend to be in the middle/upper right half of the biplot, whereas poems written by Shakespeare tend to be in the middle/lower right half. There is an overlap between poems of Eliot and Shakespeare. We infer that poems from these authors contain more characters such as for instance “r”, “u”, and “t”.

We should be careful about the inferences above since only 34 % of the total variability in data is explained by PC1 and PC2, and because there are few data points (sample size is low).

However, for these 22 poems, it seems that we can discriminate poems by Blake quite well from poems by Eliot and Shakespeare using PC1 and PC2. It seems harder to discriminate poems by Eliot from poems by Shakespeare using PC1 and PC2.

Exercise 2 - Predicting Authorship by LDA

- a) Compute the PC1- and PC2-scores of the poems in `poems.test` using the PCA from Exercise 1b).
Hint: The `predict()` function is quite useful for such purposes.

For simplicity, we only display the first 6 rows and 5 columns of `scores.test`:

##	Author	PC1	PC2	PC3	PC4
## 1	Shakespeare	3.9136323	-1.3319443	-1.1169660	1.7389198
## 2	Shakespeare	1.2103595	-0.8718919	1.4035316	1.4831095
## 3	Blake	0.3442297	0.8672537	1.9535695	1.6928408
## 4	Blake	-0.9636156	1.9237165	-0.2112698	0.2821465
## 5	Eliot	3.0454523	1.3419746	-1.6560653	1.8836301
## 6	Eliot	1.4799615	0.7610281	-0.5837241	0.2428388

If we'd like to discriminate poems between authors using PC1 and PC2 from the principal component analysis in Exercise 1, we need to predict the PC1- and PC2-scores for the 6 new poems. The `scores.test` data frame contains the predicted principal component scores for each of the 6 new poems, based on the PCA which was done on the 22 "original" poems.

- b) Conduct a linear discriminant analysis (LDA) using PC1 and PC2 from exercise 1b) as predictors to classify the six 'test' poems to either Shakespeare, Blake or Eliot.

The purpose of the linear discriminant analysis is to find combination of the variables (PC1 and PC2) that give best possible separation between groups (Blake, Eliot, Shakespeare).

First, let's take a look at `scores.poems`. For simplicity, we only display the first 6 rows and 5 columns:

##	Author	PC1	PC2	PC3	PC4
## 1	Shakespeare	1.95788838	-0.7838477	-0.08974646	0.001896807
## 2	Shakespeare	1.46252608	-0.6930659	-1.52930345	-0.242578772
## 3	Shakespeare	2.44079535	-2.0002619	2.58011089	-1.393159644
## 4	Shakespeare	3.17610723	-0.5223605	-4.22492262	-0.991816978
## 5	Shakespeare	2.07026406	-1.1138311	-0.04736645	-1.063214142
## 6	Shakespeare	0.02896731	0.6183859	3.32066984	0.184691417

This data frame contain the information about authorship and the principal component scores for each of the 22 poems.

Next, let's look at the output from the LDA:

```
lda.poems
```

```
## Call:
## lda(Author ~ PC1 + PC2, data = scores.poems)
##
## Prior probabilities of groups:
##      Blake      Eliot Shakespeare
## 0.3181818 0.3181818 0.3636364
##
## Group means:
##           PC1      PC2
## Blake      -3.062886 -0.3439229
## Eliot       1.095161  1.0599280
## Shakespeare 1.721759 -0.6265045
##
## Coefficients of linear discriminants:
##           LD1      LD2
## PC1 0.88562478 0.04059028
## PC2 0.09990848 -0.48757770
##
## Proportion of trace:
##      LD1      LD2
## 0.9665 0.0335
```

“Call” gives the linear discriminant analysis that was formulated. Here, we try to discriminate the Authors in the 22 poems based on PC1 and PC2, from the `scores.poems` data set. We have not included the 6 new poems in this analysis.

“Prior probabilities of groups” gives the prior probabilities of each group. Because we didn’t specify the prior probabilities in the `lda()` function, R uses the proportion of poems by each author as the prior probabilities.

“Group means” gives the group means in PC1 and PC2 for each author.

“Coefficients of linear discriminants”: R calculates linear discriminant functions, which are weighted sums of predictor variables. The predictor variables are PC1 and PC2, and the weights are the coefficients. For instance, $LD1 = 0.886 \cdot PC1 + 0.100 \cdot PC2$. We see that PC1 is most important in defining LD1 and PC2 is most important in defining LD2. You can read more about this in the “Additional reading”-section at the bottom of this document.

“Proportion of trace:” this is the percentage separation achieved by each linear discriminant function. About 97 % of the poems are separated by LD1, whereas the rest (3 %) are separated by LD2.

Finally, let's take a look at `lda.pred`.

Remember: `lda.pred` is created by predicting the authorship in each of the 6 new poems, using the linear discriminant analysis conducted on the 22 training poems.

```
lda.pred
```

```
## $class
## [1] Shakespeare Shakespeare Eliot      Eliot      Shakespeare Eliot
## Levels: Blake Eliot Shakespeare
##
## $posterior
```

```
##          Blake      Eliot Shakespeare
## 1 3.258286e-08 0.1601106  0.8398894
## 2 5.220699e-04 0.3851660  0.6143120
## 3 5.163776e-03 0.6218313  0.3730049
## 4 2.174979e-01 0.6276543  0.1548478
## 5 2.907146e-07 0.4186158  0.5813839
## 6 9.936598e-05 0.5108222  0.4890784
##
## $x
##          LD1          LD2
## 1 3.3329373 0.8082817
## 2 0.9848150 0.4742439
## 3 0.3915043 -0.4088812
## 4 -0.6612062 -0.9770747
## 5 2.8312027 -0.5307012
## 6 1.3867238 -0.3109883
```

“Class” gives the predicted authorship for each of the 6 poems. We see that 3 poems are classified as Shakespeare poems, 3 poems are classified as Eliot poems, and 0 poems are classified as Blake poems.

“Posterior” is a matrix whose columns are the classes (authors), rows are the individual poems, and values are the posterior probability that the corresponding poem belongs to an author.

“x” contains the linear discriminants. In general, the values are calculated using the linear discriminant functions defined in `lda.poems`, and the predicted PC scores calculated in `scores.poems` (exercise 2a).

- c) The posterior probabilities are stored in `lda.pred$posterior`. Display the values using `round(lda.pred$posterior, digits = 2)`. What is the predicted authorship of the different poems?

```
round(lda.pred$posterior, digits = 2)
```

```
##    Blake Eliot Shakespeare
## 1 0.00 0.16      0.84
## 2 0.00 0.39      0.61
## 3 0.01 0.62      0.37
## 4 0.22 0.63      0.15
## 5 0.00 0.42      0.58
## 6 0.00 0.51      0.49
```

We see that some of the poems are classified with high confidence, such as the first poem. Other poems are classified with a high level of uncertainty, such as the last poem.

- d) Write up a so-called confusion matrix for the test classification with true author as rows and classified authors as columns. Use the `confusion()` function from the package `mixlm`. What poems from which author are most difficult to predict? Compute the apparent error rate (APER) and the accuracy.

```
mixlm::confusion(poems.test$Author,      # true classes
                 lda.pred$class)         # predicted classes
```

```
##          True
## Predicted  Blake Eliot Shakespeare
```

```
## Blake      0      0      0
## Eliot      2      1      0
## Shakespeare 0      1      2
## Total      2      2      2
## Correct    0      1      2
##
## Proportions correct
##      Blake      Eliot Shakespeare
##      0.0      0.5      1.0
##
## N correct/N total = 3/6 = 0.5
```

It's most difficult to predict Blake poems. 0 % of the Blake poems were classified as Blake poems. This is noteworthy because by the 22 original poems, Blake seemed to be the easiest author to discriminate (see biplot in exercise 1d).

$$\text{APER} = \frac{\text{number of incorrectly classified poems}}{\text{total number of poems}} = \frac{3}{6} = 0.5$$

$$\text{Accuracy} = 1 - \text{APER} = 1 - 0.5 = 0.5$$

- e) How would you think one can improve the LDA? By adding more ‘test’ data (more observations in `poems.test`) or more ‘training’ data (data that the LDA is fit on, i.e. `poems`)?

Adding more test data. Try to justify this answer yourself.

Additional reading: Linear discriminant functions

The discriminant functions that you get from the `lda()` output in R (`lda.poems`) are defined as weighted sums of predictor variables.

LDA can be viewed as a variable-reduction method, same as PCA. The difference is that PCA decomposes “correlatedness” whereas LDA decomposes “separatedness”. The LDA algorithm starts by finding directions that maximize the separation between classes (authorship), then use these directions to predict the class of individuals (poems). LD1 is the strongest discriminator, LD2 is the second strongest (and so on, if there are more functions). This is also seen in the “proportion of trace” from the attached output. LD1 discriminates almost 97 % of the poems whereas LD2 discriminates the rest.

If there are k groups/classes, then $k - 1$ discriminant functions are defined.

Linear discriminant functions are used to calculate the posterior probabilities. Remember Bayes Theorem, which is used to calculate the posterior probabilities:

$$P(c_i | x_1, x_2, \dots, x_p) = c \cdot p_i \cdot f(x_1, x_2, \dots, x_n | c_i)$$

where c is a constant, p_i is the prior probability of class i , and $f(x_1, x_2, \dots, x_n | c_i)$ is the likelihood function of the observed data given class i .

With LDA, we can replace $f(x_1, x_2, \dots, x_n | c_i)$ with $g(LD_1, LD_2, \dots, LD_{k-1} | c_i)$ for an appropriate function g .